AppLocker to WDAC (App Control for Business) Migration

Implementation Solution Guide

Version 1.0.0 · 2026-06-30

Contents

Executive Summary — AppLocker to WDAC (App Control for Business) Migration 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 4

1. Solution Overview — AppLocker to WDAC (App Control for Business) Migration 4

What this migration is . . . 5

Why WDAC over AppLocker (the why — covered only here) . . . 5

The migration model . . . 5

AppLocker / WDAC coexistence . . . 5

Trust mechanisms you will use . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 6

2. Requirements Guide — AppLocker to WDAC (App Control for Business) Migration 6

2.1 Intune licensing and admin rights . . . 6

2.2 Devices enrolled in Intune . . . 7

2.3 Supported Windows OS . . . 7

2.4 The existing AppLocker policy export (the migration source) . . . 7

2.5 A WDAC authoring approach . . . 7

2.6 Managed installer (recommended trust path) . . . 7

2.7 Migration rings and pilot group . . . 7

2.8 Change control, coexistence, and rollback . . . 8

Naming convention (name every artifact ` – – `; reuse verbatim) . . . 8

Checklist . . . 8

3. Implementation Guide — AppLocker to WDAC (App Control for Business) Migration 9

Step 1 — Assess: export and inventory the current AppLocker policy . . . 9

Step 2 — Create the migration ring groups . . . 9

Step 3 — Configure: author the WDAC base policy . . . 9

Step 4 — Configure trust (translated from AppLocker) and set Audit mode . . . 9

Step 5 — Pilot: assign the audit policy and run the audit . . . 10

Step 6 — Validate the audit: analyze events and close gaps . . . 10

Step 7 — Pilot enforce: promote the pilot ring to Enforced . . . 10

Step 8 — Retire AppLocker on the pilot ring . . . 10

Step 9 — Expand: enforce and retire by ring . . . 11

5. Testing Guide — AppLocker to WDAC (App Control for Business) Migration 11

Test 1 — AppLocker baseline captured . . . 11
Test 2 — Audit policy applied . . . 11
Test 3 — Audit events are logging (no enforcement yet) . . . 11
Test 4 — Migration coverage / audit gate . . . 12
Test 5 — Enforcement active . . . 12
Test 6 — AppLocker retired, WDAC holds . . . 12
Test 7 — Tenant-wide ring/coverage report . . . 12
Pass/fail gate . . . 12
6. Troubleshooting Guide — AppLocker to WDAC (App Control for Business) Migration 12
A. Policy not applying . . . 13
B. Legitimate app would be blocked (audit) or is blocked (enforce) . . . 13
C. AppLocker still blocking after WDAC migration . . . 13
D. Over-strict enforced policy / lockout risk . . . 13
E. Managed-installer trust not taking effect . . . 14
F. Diagnostics quick reference . . . 14
G. When to escalate . . . 14
7. Best Practices — AppLocker to WDAC (App Control for Business) Migration 14
8. Deliverables Summary — AppLocker to WDAC (App Control for Business) Migration 15
Documentation . . . 15
PowerShell scripts . . . 15
What the buyer receives . . . 16
Outcome . . . 16
Appendix A — PowerShell Scripts 17

Executive Summary — AppLocker to WDAC (App Control for Business) Migration

AppLocker has served its time, but it enforces application control late in the boot/run chain and is increasingly outpaced by modern threats. WDAC — now branded App Control for Business — enforces deeper, at the OS code-integrity layer, and aligns with current security practice. The risk in switching is real: get a WDAC policy wrong and you lock users out of the apps they need. This solution is the controlled migration from AppLocker to WDAC, delivered through Microsoft Intune.

This is a complete, deploy-it-today package for migrating application control off AppLocker and onto WDAC / App Control for Business, written for an administrator with no prior WDAC experience.

What you get

- A plain-language Solution Overview of why WDAC beats AppLocker, the migration model (translate to audit to enforce), AppLocker/WDAC coexistence, and managed-installer / ISG trust.
- A Requirements Guide with verify / implement-if-missing for every prerequisite, including the existing AppLocker policy export and a pilot ring.
- An Implementation Guide of precise, ordered Intune steps: export and translate AppLocker rules, author the WDAC policy, deploy in audit mode, close gaps, enforce by ring, and retire AppLocker.
- Five PowerShell scripts for current-state discovery, audit-event and policy validation, a migration coverage and would-block report, safe audit-mode revert, and a migration-ring group builder.
- Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

A WDAC policy translated from your existing AppLocker rules, deployed first in audit mode so it catches every gap before it can block anyone, then promoted to enforcement in rings — pilot first, fleet last — with AppLocker kept coexisting until the WDAC policy is proven, then retired. No user lockouts: enforcement is never turned on until the audit log is clean of legitimate blocks.

The deployment model

Assess to Configure to Pilot to Validate to Expand — with a mandatory audit gate before any enforcement and a documented rollback at every stage.

Outcome

Application control moved from AppLocker to WDAC — translated, audited, and enforced in rings with no user lockouts — and AppLocker retired, proven on a pilot before you scale.

1. Solution Overview — AppLocker to WDAC (App Control for Business) Migration

What this migration is

This is the controlled replacement of `AppLocker` with `WDAC` (Windows Defender Application Control, now branded App Control for Business) as your Windows application-control mechanism, delivered and assigned through Microsoft Intune. You are not standing up app control for the first time — you already run AppLocker — so the work is a migration: translate what AppLocker currently allows into a WDAC policy, prove that policy in audit mode, enforce it in rings, and only then retire AppLocker.

Why WDAC over AppLocker (the why — covered only here)

- OS-level enforcement. WDAC is enforced by the kernel code-integrity engine, earlier and deeper than AppLocker's user-mode service, so it is far harder to bypass.
- Modern trust model. WDAC trusts code by signer, hash, file attributes, a managed installer, or the Intelligent Security Graph (ISG) reputation service — richer and more durable than AppLocker's publisher/path/hash rules, which break on every app update.
- Strategic direction. Microsoft positions App Control for Business as the forward path for Windows application control; AppLocker is maintained but no longer the recommended primary control.
- Single managed surface. Both the WDAC policy and the migration rings are authored and assigned from Intune, giving one console, one audit trail, and one assignment model.

The migration model

1. Translate. Export the existing AppLocker policy and inventory what it allows, then author an equivalent WDAC base policy (trust Windows + Store, plus managed installer and optionally ISG, plus any explicit signer/hash rules carried over from AppLocker).
1. Audit. Deploy the WDAC policy in audit mode via Intune. Audit mode blocks nothing — it only logs what would have been blocked (Code Integrity event 3076), so no user is ever locked out during this phase.
1. Analyze and close gaps. Read the audit events, identify legitimate apps that would have been blocked, and add trust for them (preferably by deploying them through Intune so the managed installer covers them). Re-audit until the log is clean of legitimate blocks.
1. Enforce by ring. Promote the policy to enforced for the pilot ring first, validate, then expand to broad rings in waves.
1. Retire AppLocker. Once WDAC enforcement is proven across the fleet, remove the AppLocker policy.

AppLocker / WDAC coexistence

AppLocker and WDAC can run on the same device at the same time — they are independent engines and both evaluate execution. During this migration they coexist deliberately: AppLocker stays in force while the

WDAC policy runs in audit mode and through early enforcement rings, so application control is never absent mid-cutover. AppLocker is removed only as the final step, per ring, after WDAC enforcement is validated. Where both enforce, the most restrictive outcome wins, so the audit phase is essential to confirm WDAC trusts everything AppLocker allowed before AppLocker is withdrawn.

Trust mechanisms you will use

• Managed installer. Apps deployed through Intune are tagged as installed by a trusted managed installer, so their files are automatically trusted by WDAC — the primary way to keep business apps allowed without per-app rules.

• ISG (Intelligent Security Graph). Optional reputation-based trust for well-known, good-reputation apps not deployed through Intune.

• Carried-over rules. Explicit signer or hash rules translated from AppLocker for apps that are neither Intune-deployed nor ISG-trusted.

Limitations (be honest with buyers)

• WDAC is stricter than AppLocker by design; unsigned or low-reputation line-of-business apps that AppLocker allowed by path must be re-trusted by signer, hash, managed installer, or ISG — this is exactly what the audit phase surfaces.

• Some kernel-level WDAC features (for example HVCI) interact with hardware/driver compatibility; this migration uses application (user-mode) control and does not require enabling HVCI.

• A misconfigured enforced policy can block boot or critical apps — which is why this solution never enforces before the audit log is clean and always keeps a rollback path.

The standard deployment model

> *Translate AppLocker rules, author the WDAC policy, deploy in audit mode, close gaps, enforce by ring, retire AppLocker. The pattern: Assess to Configure to Pilot to Validate to Expand.*

2. Requirements Guide — AppLocker to WDAC (App Control for Business) Migration

Verify every requirement before configuring. Each entry: Description · Why · Verify · Implement-if-missing. Category-wide prerequisites are in `global-requirements-migrations.md`.

2.1 Intune licensing and admin rights

Description: An Intune-bearing license plan (Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1) and the Intune Administrator role. WDAC itself is a free Windows feature. Why: Intune authors and assigns the WDAC audit/enforce policies and the migration-ring groups. Verify: `https://intune.microsoft.com` loads; Tenant administration to Tenant status is active; you hold

Intune Administrator. Implement: Assign an Intune-bearing plan and grant least-privilege admin roles.

2.2 Devices enrolled in Intune

Description: The in-scope Windows devices are enrolled in Intune. Why: WDAC policies and ring assignments apply only to enrolled devices. Verify: Intune to Devices to All devices; run `WDACMIG-Discovery.ps1`. Implement: Enable automatic enrollment; enroll the in-scope devices.

2.3 Supported Windows OS

Description: Windows 10 1903+ or Windows 11 (Pro/Enterprise/Education). Windows 11 adds `citool` for local policy inspection. Why: App Control for Business policy deployment and the Code Integrity audit log require a supported build. Verify: `winver`; run `WDACMIG-Discovery.ps1` (reports OS support verdict). Implement: Patch/upgrade to a supported build before migrating that device.

2.4 The existing AppLocker policy export (the migration source)

Description: A current export of the in-force AppLocker policy from the devices being migrated. Why: The WDAC policy is translated from what AppLocker currently allows; without the export you cannot guarantee parity and risk locking out apps AppLocker permitted. Verify: Run `WDACMIG-Discovery.ps1` (captures the effective AppLocker policy and rule collections), or `Get-AppLockerPolicy -Effective -Xml` on a representative device. Implement: Export the effective AppLocker policy from representative devices in each ring before authoring WDAC.

2.5 A WDAC authoring approach

Description: A decided method for producing the WDAC base policy — the Intune App Control for Business profile (built-in trust: Windows + Store + managed installer + optional ISG) for most fleets, with the WDAC Wizard or `New-CIPolicy` for any custom signer/hash rules carried over from AppLocker. Why: You must produce a concrete policy artifact to deploy in audit mode; the built-in Intune profile covers the common case, custom policy covers carried-over AppLocker specifics. Verify: You can open Intune to Endpoint security to App Control for Business and create a policy, and (if needed) you have the WDAC Wizard installed for custom XML. Implement: Use the Intune built-in profile for trust; author supplemental/custom XML only for apps not covered by managed installer or ISG.

2.6 Managed installer (recommended trust path)

Description: Intune configured as a managed installer so apps deployed through Intune are automatically trusted by WDAC. Why: The managed installer is the lowest-maintenance way to keep business apps allowed after enforcement, replacing AppLocker's brittle path/publisher rules. Why it matters for migration: apps AppLocker allowed by path are best re-trusted by redeploying them through Intune under the managed installer rather than re-creating path rules in WDAC. Verify: Intune to Endpoint security to App Control for Business — the managed installer setting is enabled; `WDACMIG-Discovery.ps1` reports managed-installer tagging where present. Implement: Enable the managed installer in the App Control for Business settings and deploy line-of-business apps through Intune.

2.7 Migration rings and pilot group

Description: Microsoft Entra security groups for the migration rings, named per the naming convention below, with a pilot ring populated first. Why: Enforcement is rolled out in rings (pilot to broad) so a too-strict policy is caught on a small group before the fleet. Verify: Entra to Groups; the ring groups exist; `WDACMIG-Set-MigrationRingTargeting.ps1` can populate them. Run `WDACMIG-Report.ps1` to confirm ring coverage. Implement: Create the ring groups (see naming) and add pilot devices first.

2.8 Change control, coexistence, and rollback

Description: A change window, the documented AppLocker/WDAC coexistence plan (AppLocker stays in force until WDAC enforcement is validated per ring), and a rollback (unassign WDAC enforce, revert to audit) before any enforcement. Why: Enforcement touches live production devices; cutover must be reversible and coordinated. Verify: A rollback procedure and coexistence plan are written and approved before Step 8 (enforce). Implement: Define the audit-to-enforce gate, the per-ring rollback, and the order coexistence to enforce to AppLocker retirement.

Naming convention (name every artifact `<Purpose> – <Scope> – <Stage>`; reuse verbatim)

Artifact	Name
WDAC base policy	`App Control – Migrated from AppLocker – Base`
Audit profile (Intune)	`App Control – Fleet – Audit`
Enforce profile (Intune)	`App Control – Fleet – Enforce`
Pilot ring group	`App Control Migration – Pilot – Ring 1`
Broad ring group	`App Control Migration – Broad – Ring 2`
Fleet ring group	`App Control Migration – Fleet – Ring 3`

Checklist

- Intune licensing + Intune Administrator
- Devices enrolled
- Windows 10 1903+ / Windows 11 — `WDACMIG-Discovery.ps1`
- Effective AppLocker policy exported — `WDACMIG-Discovery.ps1`
- WDAC authoring approach decided
- Managed installer enabled; LOB apps via Intune
- Migration ring groups created (Pilot / Broad / Fleet)
- Coexistence + audit-to-enforce gate + rollback approved

3. Implementation Guide — AppLocker to WDAC (App Control for Business) Migration

Follow these steps in order, in the Microsoft Intune admin center (`https://intune.microsoft.com`). Use the artifact names from the Requirements Guide verbatim.

> *Model: translate AppLocker rules, author the WDAC policy, deploy audit-first via Intune, close gaps, enforce by ring, retire AppLocker. AppLocker stays in force throughout audit and early enforcement (coexistence) and is removed last. No enforcement until the audit log is clean — no lockouts.*

Step 1 — Assess: export and inventory the current AppLocker policy

1. On a representative device in each ring, run `WDACMIG-Discovery.ps1` (captures the effective

AppLocker policy, its rule collections, current WDAC/code-integrity state, and managed-installer status). It writes the effective AppLocker XML for review.

1. Inventory what AppLocker allows: signers/publishers, file paths, and hash rules. Flag path-based

rules for unsigned line-of-business apps — these are the apps that must be re-trusted under WDAC by managed installer, ISG, or explicit signer/hash rules.

Step 2 — Create the migration ring groups

1. Entra to Groups to New group to Security. Create `App Control Migration – Pilot – Ring 1` and add a

few pilot devices. Create `App Control Migration – Broad – Ring 2` and `App Control Migration – Fleet – Ring 3` for later waves.

1. Optionally populate the pilot ring with `WDACMIG-Set-MigrationRingTargeting.ps1`.

Step 3 — Configure: author the WDAC base policy

1. Intune to Endpoint security to App Control for Business to Create Policy.

(If not present in your tenant, use Devices to Configuration to Create to Settings catalog and search "App Control" / "Application Control".)

1. Platform: Windows. Profile: App Control for Business. Create.
2. Name: `App Control – Fleet – Audit`. Next.

Step 4 — Configure trust (translated from AppLocker) and set Audit mode

On Configuration settings:

1. Application control code integrity policies = Enable.
2. Trust mode / built-in policy = Trust Windows components and Store apps.
3. Trust apps from managed installers = Enable (so Intune-deployed apps — the re-trust path for

AppLocker's path-allowed apps — are allowed).

1. Also trust apps with good reputation (ISG) = Enable (recommended for good-reputation apps not deployed via Intune).

1. For apps AppLocker allowed that are neither Intune-deployed nor ISG-trusted, plan a supplemental/custom policy carrying the translated signer/hash rules (author with the WDAC Wizard or `New-CIPolicy`); reference it as `App Control – Migrated from AppLocker – Base`.

1. Enforcement = Audit only.

Click Next.

Step 5 — Pilot: assign the audit policy and run the audit

1. Assignments to Add groups to `App Control Migration – Pilot – Ring 1`. Next to Create.

2. Keep AppLocker in force on these devices (coexistence) — do not remove it yet.

3. Let pilot devices run normally for 2+ weeks, exercising all line-of-business apps. Audit mode blocks nothing; it only logs would-block events (Code Integrity 3076).

Step 6 — Validate the audit: analyze events and close gaps

1. On pilot devices run `WDACMIG-Validation.ps1` (confirms the audit policy is applied and the device is logging) and `WDACMIG-Report.ps1` (summarizes would-block events and migration coverage — which AppLocker-allowed apps WDAC would block).

1. For each legitimate app that would be blocked:

• Preferred: deploy it via Intune so the managed installer trust covers it.

• For signed apps not from Intune, rely on ISG, or add the translated signer/hash rule to the supplemental policy `App Control – Migrated from AppLocker – Base`.

1. Re-audit until `WDACMIG-Report.ps1` shows the audit log is clean of legitimate blocks. This is the audit gate — do not enforce before it passes.

Step 7 — Pilot enforce: promote the pilot ring to Enforced

1. Duplicate the policy as `App Control – Fleet – Enforce` (or edit) and set Enforcement = Enforced.

Save. Keep it assigned to `App Control Migration – Pilot – Ring 1`.

1. Re-validate with `WDACMIG-Validation.ps1` (confirms an enforced policy is active).

2. Confirm pilot users can run all required apps; untrusted code is blocked. AppLocker still coexists.

Step 8 — Retire AppLocker on the pilot ring

1. Only after Step 7 validates, remove the AppLocker policy from the pilot devices (clear the

AppLocker rule collections / disable the Application Identity policy via your AppLocker management method).

1. Re-run `WDACMIG-Validation.ps1` to confirm WDAC enforcement remains active with AppLocker gone.

Step 9 — Expand: enforce and retire by ring

1. Repeat Steps 5 to 8 per ring: assign `App Control – Fleet – Audit` to the next ring, run the audit,

close gaps, promote to `App Control – Fleet – Enforce`, validate, then retire AppLocker on that ring. Roll out `App Control Migration – Broad – Ring 2` then `App Control Migration – Fleet – Ring 3` in waves.

1. Keep new apps flowing through Intune (managed installer) so they stay trusted.
2. Re-validate each ring with `WDACMIG-Validation.ps1` and `WDACMIG-Report.ps1`.

> *Recovery / rollback: if an enforced policy is too strict, unassign* `App Control – Fleet – Enforce` *in Intune, re-assign* `App Control – Fleet – Audit`*, and run* `WDACMIG-Remediation.ps1` *(default -WhatIf) to revert the device to audit mode. If AppLocker was already retired on that ring, re-deploy it temporarily for coverage while you fix WDAC trust, then re-enforce.*

Order of operations

`Export AppLocker (assess)` → `Ring groups` → `Author WDAC (Audit)` → `Assign pilot + coexist` → `Review audit + close gaps (audit gate)` → `Enforce pilot` → `Retire AppLocker (pilot)` → `Expand ring by ring`

5. Testing Guide — AppLocker to WDAC (App Control for Business) Migration

Run after each ring's audit policy applies (Implementation Step 5) and again after enforcement (Step 7) and after AppLocker retirement (Step 8).

Test 1 — AppLocker baseline captured

Do: Run `WDACMIG-Discovery.ps1` on a representative device in the ring. Expected: The effective AppLocker policy and its rule collections are reported and exported; current WDAC state and managed-installer status are shown.

Test 2 — Audit policy applied

Do: Intune to Endpoint security to App Control for Business to `App Control – Fleet – Audit` to Device status; on a pilot device run `WDACMIG-Validation.ps1`. Expected: Pilot devices show Succeeded; the script reports `WDAC ACTIVE (Audit)`.

Test 3 — Audit events are logging (no enforcement yet)

Do: Exercise line-of-business apps on a pilot device, then run `WDACMIG-Report.ps1 -Days 14`. Expected: Would-block events (Code Integrity 3076) are captured; nothing is actually blocked; apps all run because AppLocker still coexists and WDAC is audit-only.

Test 4 — Migration coverage / audit gate

Do: Run `WDACMIG-Report.ps1` after closing gaps. Expected: The would-block report is clean of legitimate apps (only genuinely unwanted code remains). This is the gate to enforce.

Test 5 — Enforcement active

Do: After promoting to `App Control – Fleet – Enforce`, run `WDACMIG-Validation.ps1`. Expected: The script reports `WDAC ACTIVE (Enforced)`; required apps run; untrusted code is blocked (Code Integrity 3077).

Test 6 — AppLocker retired, WDAC holds

Do: After removing AppLocker on the ring (Step 8), run `WDACMIG-Validation.ps1` and `WDACMIG-Discovery.ps1`. Expected: AppLocker rule collections are gone; WDAC remains `ACTIVE (Enforced)`; apps still run.

Test 7 — Tenant-wide ring/coverage report

Do: Run `WDACMIG-Report.ps1` (tenant/Graph mode) to list App Control policies and assigned ring groups. Expected: The audit and enforce profiles are assigned to the expected ring groups, in the expected order.

Pass/fail gate

Gate	Requirement
Audit gate (must pass before enforce)	Tests 1–4 pass; `WDACMIG-Report.ps1` shows no legitimate would-blocks
Pilot pass	Tests 5–6 pass on `App Control Migration – Pilot – Ring 1`
Ring expand pass	Tests 2–6 pass for the ring before retiring AppLocker on it

6. Troubleshooting Guide — AppLocker to WDAC (App Control for Business) Migration

Use `WDACMIG-Discovery.ps1` / `WDACMIG-Validation.ps1` for device state, `WDACMIG-Report.ps1` for audit events and tenant coverage, and `WDACMIG-Remediation.ps1` (default -WhatIf) for a safe revert to audit mode.

A. Policy not applying

Symptom	Cause	Remediation
No WDAC policy on device	Not assigned, not synced, or wrong group	Confirm `App Control – Fleet – Audit` is assigned to the device's ring group; sync; run `WDACMIG-Validation.ps1`
`App Control – Fleet – Audit` shows Error	Device not enrolled / not checked in	Confirm enrollment and ring-group membership; sync
Windows 10 shows no policy via citool	`citool` is Windows 11 only	Use `WDACMIG-Validation.ps1` (DeviceGuard WMI) for Windows 10 state

B. Legitimate app would be blocked (audit) or is blocked (enforce)

• Audit phase (Code Integrity 3076): expected — that is the point. Run `WDACMIG-Report.ps1`, then add

trust: deploy the app via Intune (managed installer), rely on ISG, or add a translated signer/hash rule to `App Control – Migrated from AppLocker – Base`. Re-audit until clean.

• Enforce phase (Code Integrity 3077, app actually blocked): you enforced before the audit gate was

clean. Roll back: unassign `App Control – Fleet – Enforce`, re-assign `App Control – Fleet – Audit`, run `WDACMIG-Remediation.ps1`, fix trust, then re-enforce.

C. AppLocker still blocking after WDAC migration

Symptom	Cause	Remediation
App blocked even though WDAC trusts it	AppLocker (coexisting) still denies it — most restrictive wins	This is expected during coexistence; once WDAC enforcement is validated on the ring, retire AppLocker (Step 8); until then, add an AppLocker allow rule or accept the block
Apps break right after AppLocker retirement	A gap WDAC did not cover that AppLocker had	Re-deploy AppLocker temporarily for coverage, run `WDACMIG-Report.ps1`, close the WDAC gap, then retire AppLocker again

D. Over-strict enforced policy / lockout risk

• Unassign `App Control – Fleet – Enforce` in Intune, re-assign `App Control – Fleet – Audit`, and run

`WDACMIG-Remediation.ps1` (omit -WhatIf to apply) to revert the device to audit mode. A reboot may be required to fully clear a persistent policy.

- If the device cannot boot to a usable state, boot to recovery and remove the active policy from `%WINDIR%\System32\CodeIntegrity\CiPolicies\Active` (advanced), then re-sync.

E. Managed-installer trust not taking effect

- Confirm the managed installer is enabled in the App Control for Business settings and the app was installed by Intune after that setting applied — files installed before tagging are not retroactively trusted. Re-deploy the app via Intune.

F. Diagnostics quick reference

```
Get-AppLockerPolicy -Effective -Xml                                    # current AppLocker policy
Get-CimInstance -Namespace root\Microsoft\Windows\DeviceGuard -ClassName Win32_DeviceGuard | Select CodeIn
& "$env:WINDIR\System32\citool.exe" --list-policies                    # Windows 11 active policies
Get-WinEvent -FilterHashtable @{ LogName='Microsoft-Windows-CodeIntegrity/Operational'; Id=3076,3077 } -Ma
Start-Process "$env:WINDIR\System32\deviceenroller.exe" -ArgumentList "/c /AutoEnrollMDM"   # MDM sync
```

G. When to escalate

If the policy shows applied, the audit log is clean, the device is synced, and a legitimate app is still blocked under enforcement after adding managed-installer/ISG/signer trust, collect the Code Integrity 3077 events (`WDACMIG-Report.ps1 -CsvPath`) and the policy XML and open a Microsoft support case.

7. Best Practices — AppLocker to WDAC (App Control for Business) Migration

- Audit first, always. Never enforce a WDAC policy before its audit log is clean of legitimate would-blocks. The audit gate is the single most important practice — it is how you migrate with no user lockouts.
- Translate, do not guess. Author the WDAC policy from the exported AppLocker policy, not from memory.

Pay special attention to AppLocker path rules for unsigned apps — those are the highest lockout risk under WDAC.

- Coexist on purpose. Keep AppLocker in force while WDAC runs in audit and through early enforcement,

so application control is never absent. Retire AppLocker per ring, only after WDAC enforcement is validated on that ring.

- Prefer the managed installer. Re-trust line-of-business apps by deploying them through Intune under the managed installer rather than re-creating brittle path rules — it survives app updates and replaces AppLocker's weakest construct.
- Use ISG for reputable apps. Enable Intelligent Security Graph trust for well-known,

good-reputation apps that are not deployed through Intune, to shrink the supplemental rule set.

• Ring everything. Pilot to broad to fleet. A too-strict policy caught on `App Control Migration – Pilot – Ring 1` never reaches the fleet.

• Name artifacts consistently. Use the `<Purpose> – <Scope> – <Stage>` names verbatim across audit profile, enforce profile, base policy, and ring groups so audits and handoffs are unambiguous.

• Keep a rollback ready. Pre-plan the unassign-enforce / re-assign-audit path and keep `WDACMIG-Remediation.ps1` (default -WhatIf) on hand. If AppLocker is already retired on a ring, know how to re-deploy it temporarily.

• Re-audit after change. Any new or updated app can introduce a would-block; re-run `WDACMIG-Report.ps1` after notable app changes, not just at migration time.

• Document the cutover. Record per-ring dates, the audit-gate sign-off, enforcement, and AppLocker-retirement for audits and repeatability.

8. Deliverables Summary — AppLocker to WDAC (App Control for Business) Migration

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	Why WDAC beats AppLocker, the translate-to-audit-to-enforce model, coexistence, managed installer / ISG
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing, plus the naming convention
Global Requirements (Migrations)	Category-wide migration prerequisites, published once
3. Implementation Guide	Exact, ordered Intune steps: export AppLocker, author WDAC, audit, close gaps, enforce by ring, retire AppLocker
5. Testing Guide	Validation procedures with the audit gate and pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`WDACMIG-Discovery.ps1`	Captures the effective AppLocker policy + current WDAC/code-integrity and managed-installer state (read-only)
`WDACMIG-Validation.ps1`	Confirms the WDAC policy is applied and reports Audit vs Enforced (read-only)
`WDACMIG-Report.ps1`	Migration coverage + would-block report from Code Integrity audit events, and tenant policy/ring assignments via Graph (read-only)
`WDACMIG-Remediation.ps1`	Safe revert to audit mode after unassigning the enforce policy; default -WhatIf
`WDACMIG-Set-MigrationRingTargeting.ps1`	Builds/populates the migration ring groups and assigns the audit/enforce profiles (Microsoft Graph)

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process: Assess to Configure to Pilot to Validate to Expand, with a mandatory audit gate before enforcement and rollback at every stage.

Outcome

Application control moved from AppLocker to WDAC — translated, audited, and enforced in rings with no user lockouts — and AppLocker retired, proven on a pilot before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

WDACMIG-Discovery.ps1

Purpose: Reads the device's current application-control posture before migration: the effective AppLocker policy and its rule collections (the migration source), the OS support verdict for App Control for Business, the current code-integrity enforcement state, any active App Control policies, and managed-installer status. Read-only. HOW TO RUN Elevated: powershell -ExecutionPolicy Bypass -File .\WDACMIG-Discovery.ps1 [-XmlPath C:\Temp\applocker.xml] EXPECTED RESULTS AppLocker rule-collection summary (exported to XML if -XmlPath given), OS support verdict, current WDAC enforcement state, and active policy list — the baseline for translation. .NOTES Read-only. Run on a representative device in each migration ring. #> [CmdletBinding()] param([string]$XmlPath)

How to run: Elevated: powershell -ExecutionPolicy Bypass -File .\WDACMIG-Discovery.ps1 [-XmlPath C:\Temp\applocker.xml] EXPECTED RESULTS AppLocker rule-collection summary (exported to XML if -XmlPath given), OS support verdict, current WDAC enforcement state, and active policy list — the baseline for translation. .NOTES Read-only. Run on a representative device in each migration ring. #> [CmdletBinding()] param([string]$XmlPath)

WDACMIG-Remediation.ps1

Purpose: Recovery for an over-strict enforced WDAC policy during migration. After you UNASSIGN 'App Control - Fleet - Enforce' in Intune (and re-assign 'App Control - Fleet - Audit'), this forces an MDM sync so the change reaches the device and (Windows 11) refreshes the local policy set with citool, returning the device to a non-blocking audit state. Runs in -WhatIf by default — it only previews actions until you pass -WhatIf:$false. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\WDACMIG-Remediation.ps1 Apply: powershell -ExecutionPolicy Bypass -File .\WDACMIG-Remediation.ps1 -WhatIf:$false EXPECTED RESULTS With -WhatIf (default): the steps it WOULD take. Applied: MDM sync requested and local policy refreshed; reboot if a policy persists, then run WDACMIG-Validation.ps1 (expect Audit). .NOTES Unassign the Intune Enforce policy and re-assign Audit FIRST. Default -WhatIf for safety. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param()

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\WDACMIG-Remediation.ps1 Apply: powershell -ExecutionPolicy Bypass -File .\WDACMIG-Remediation.ps1 -WhatIf:$false EXPECTED RESULTS With -WhatIf (default): the steps it WOULD take. Applied: MDM sync requested and local policy refreshed; reboot if a policy persists, then run WDACMIG-Validation.ps1 (expect Audit). .NOTES Unassign the Intune Enforce policy and re-assign Audit FIRST. Default -WhatIf for safety. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param()

WDACMIG-Report.ps1

Purpose: Reads the Code Integrity Operational log for events 3076 (audit / would-block) and 3077 (enforced block) and summarizes the files/publishers involved, so you can decide what to add to trust BEFORE enforcing — this is the audit gate. Optionally also reports tenant-wide App Control policies and their ring-group assignments via Microsoft Graph. Read-only. HOW TO RUN Device audit report

(elevated): powershell -ExecutionPolicy Bypass -File .\WDACMIG-Report.ps1 -Days 14 [-CsvPath C:\Temp\wdacmig.csv] Add tenant policy/assignment report: powershell -ExecutionPolicy Bypass -File .\WDACMIG-Report.ps1 -Tenant (requires Microsoft Graph SDK and DeviceManagementConfiguration.Read.All) EXPECTED RESULTS A breakdown of would-block files by type/path/publisher (review for legitimate apps to trust); a clean report = the audit gate is met. With -Tenant, the audit/enforce profiles and their assigned ring groups. .NOTES Read-only. #> [CmdletBinding()] param([int]$Days = 14, [string]$CsvPath, [switch]$Tenant)

How to run: Device audit report (elevated): powershell -ExecutionPolicy Bypass -File .\WDACMIG-Report.ps1 -Days 14 [-CsvPath C:\Temp\wdacmig.csv] Add tenant policy/assignment report: powershell -ExecutionPolicy Bypass -File .\WDACMIG-Report.ps1 -Tenant (requires Microsoft Graph SDK and DeviceManagementConfiguration.Read.All) EXPECTED RESULTS A breakdown of would-block files by type/path/publisher (review for legitimate apps to trust); a clean report = the audit gate is met. With -Tenant, the audit/enforce profiles and their assigned ring groups. .NOTES Read-only. #> [CmdletBinding()] param([int]$Days = 14, [string]$CsvPath, [switch]$Tenant)

WDACMIG-Set-MigrationRingTargeting.ps1

Purpose: Creates the migration-ring Entra security groups (Pilot / Broad / Fleet) if missing, optionally adds devices to a ring, and assigns the App Control audit/enforce configuration profiles to a ring group via Microsoft Graph — so ring targeting is consistent and repeatable. This script WRITES to Entra/Intune; preview first with -WhatIf. HOW TO RUN Create the ring groups (preview): powershell -ExecutionPolicy Bypass -File .\WDACMIG-Set-MigrationRingTargeting.ps1 -WhatIf Create groups for real and add device IDs to the pilot ring: powershell -ExecutionPolicy Bypass -File .\WDACMIG-Set-MigrationRingTargeting.ps1 -Ring Pilot -DeviceIds ,<...> -WhatIf:$false Assign the audit profile to the pilot ring: powershell -ExecutionPolicy Bypass -File .\WDACMIG-Set-MigrationRingTargeting.ps1 -Ring Pilot -AssignProfile 'App Control - Fleet - Audit' -WhatIf:$false Requires Microsoft Graph SDK and scopes: Group.ReadWrite.All, DeviceManagementConfiguration.ReadWrite.All. EXPECTED RESULTS The named ring group(s) exist; requested devices are members; and (if -AssignProfile) the named App Control profile is assigned to the ring group. With -WhatIf, only the planned actions. .NOTES Writes to Entra/Intune. Default behavior follows -WhatIf; pass -WhatIf:$false to apply. #> [CmdletBinding(SupportsShouldProcess = $true)] param([ValidateSet('Pilot','Broad','Fleet')] [string]$Ring, [string[]]$DeviceIds, [string]$AssignProfile)

How to run: Create the ring groups (preview): powershell -ExecutionPolicy Bypass -File .\WDACMIG-Set-MigrationRingTargeting.ps1 -WhatIf Create groups for real and add device IDs to the pilot ring: powershell -ExecutionPolicy Bypass -File .\WDACMIG-Set-MigrationRingTargeting.ps1 -Ring Pilot -DeviceIds <entraDeviceObjectId>,<...> -WhatIf:$false Assign the audit profile to the pilot ring: powershell -ExecutionPolicy Bypass -File .\WDACMIG-Set-MigrationRingTargeting.ps1 -Ring Pilot -AssignProfile 'App Control - Fleet - Audit' -WhatIf:$false Requires Microsoft Graph SDK and scopes: Group.ReadWrite.All, DeviceManagementConfiguration.ReadWrite.All. EXPECTED RESULTS The named ring group(s) exist; requested devices are members; and (if -AssignProfile) the named App Control profile is assigned to the ring group. With -WhatIf, only the planned actions. .NOTES Writes to Entra/Intune. Default behavior follows -WhatIf; pass -WhatIf:$false to apply. #> [CmdletBinding(SupportsShouldProcess = $true)] param([ValidateSet('Pilot','Broad','Fleet')] [string]$Ring, [string[]]$DeviceIds, [string]$AssignProfile)

WDACMIG-Validation.ps1

Purpose: Verifies that a WDAC (App Control for Business) code-integrity policy is active on the device and whether it is in Audit or Enforced mode, and confirms the Code Integrity Operational log is present so audit events are being recorded. Use after Implementation Steps 5, 7, and 8. HOW TO RUN Elevated: powershell -ExecutionPolicy Bypass -File .\WDACMIG-Validation.ps1 EXPECTED RESULTS "WDAC ACTIVE (Audit)" during the audit phase, "WDAC ACTIVE (Enforced)" after enforcement, or "NO ACTIVE POLICY" if none applied (check assignment / sync). Read-only. .NOTES Read-only. Run on a device in the ring being migrated. #> [CmdletBinding()] param() Write-Host "=== AppLocker to WDAC Migration — Validation ($($env:COMPUTERNAME)) ===" -ForegroundColor Cyan

How to run: Elevated: powershell -ExecutionPolicy Bypass -File .\WDACMIG-Validation.ps1 EXPECTED RESULTS "WDAC ACTIVE (Audit)" during the audit phase, "WDAC ACTIVE (Enforced)" after enforcement, or "NO ACTIVE POLICY" if none applied (check assignment / sync). Read-only. .NOTES Read-only. Run on a device in the ring being migrated. #> [CmdletBinding()] param() Write-Host "=== AppLocker to WDAC Migration — Validation ($($env:COMPUTERNAME)) ===" -ForegroundColor Cyan